

BBD2 Parcial — 1era fecha 10/06/2026

Enunciado general

El siguiente diagrama modela un repositorio de publicaciones.

Documento es una clase abstracta de la que heredan **Libro** y **Tesis**. Un Documento es escrito por uno o más Autores y un Autor puede escribir muchos Documentos —relación “**escrito por**”, muchos-a-muchos—.

Cada Documento se compone de uno o más capítulos —relación “**contiene**”, composición—.

La mayoría de las consultas del modelo se realizan sobre los documentos y los capítulos relacionados a este.

Restricciones de atributos

Autor:

`id: Long`, `email: String` —no puede repetirse para distintos autores—,
`nombreCompleto: String`, `afiliacion: String`

Documento abstracto:

`id: Long`, `titulo: String`, `anioPublicacion: int`, `idioma: String`

Libro:

`isbn: String` —no puede repetirse para distintos libros—, `numeroEdicion: int`,
`genero: String`

Tesis:

`universidad: String`, `director: String`, `carrera: String`, `notaDefensa: Double`

Capitulo:

`titulo: String`, `numeroOrden: int`, `cantidadPaginas: int`

Hibernate / JPA

1.

Al mapear la jerarquía **Documento / Libro / Tesis** a tablas relacionales, ¿qué estrategia de herencia de las posibles opciones de Hibernate/JPA utilizaría? Justificar la decisión teniendo en cuenta que las subclases poseen atributos obligatorios propios —por ejemplo *isbn* en Libro y *universidad* en Tesis— y relaciones propias.

Seleccione la única opción correcta:

- a. *SINGLE_TABLE*
 - b. *JOINED*
 - c. *TABLE_PER_CLASS*
 - d. No es posible mapear esta jerarquía con JPA porque las subclases tienen relaciones propias y se debe eliminar la herencia y duplicar los atributos comunes.
-

2.

Definir el mapeo JPA completo de las tres entidades de la jerarquía —Documento, Libro y Tesis— según el diagrama.

Tener en cuenta las restricciones de los atributos comunes.

Incluir las anotaciones JPA sobre la siguiente estructura de clases, en los renglones indicados. Puede agregar los renglones que necesite.

```
public abstract class Documento {  
  
    // ... atributos ya mapeados (id, titulo, anioPublicacion, idioma)  
  
    // getters y setters omitidos  
}  
  
public class Libro extends Documento {  
  
    private String isbn;  
    private int numeroEdicion;  
    private String genero;  
  
    // getters y setters omitidos  
}  
  
public class Tesis extends Documento {  
  
    private String universidad;  
    private String director;  
    private String carrera;  
    private Double notaDefensa;
```

```
// getters y setters omitidos
}
```

3.

Analizar la relación “**contiene**” entre Documento y Capitulo y responda las siguientes preguntas:

- a. ¿Qué anotaciones se utilizan en cada lado? ¿Dónde se ubica **mappedBy** y en qué tabla aparece la clave foránea?
- b. ¿Qué **FetchType** elegiría en cada extremo de la relación? Justificar.
- c. ¿Qué operaciones en cascada —y **orphanRemoval**— configuraría desde Documento hacia Capitulo? Justificar a partir de la semántica de composición.

```
public abstract class Documento {

    // ... atributos ya mapeados (id, titulo, anioPublicacion, idioma)

    private List<Capitulo> capitulos = new ArrayList<>();

    // getters y setters
}

public class Capitulo {

    // ... atributos ya mapeados (id, titulo, numeroOrden, cantidadPaginas)

    private Documento documento;

    // getters y setters
}
```

Spring Data JPA

4.

Sobre la jerarquía de repositorios y las consultas en Spring Data JPA, ¿cuáles de las siguientes afirmaciones son correctas? Puede haber más de una.

Seleccione todas las opciones correctas:

- a. `CrudRepository` provee las operaciones básicas (guardar, actualizar, eliminar, obtener todos) sin necesidad de implementarlas, esto gracias a que hereda de `Repository`
 - b. Un `Query Method` deriva la consulta a partir del nombre del método y sus parámetros.
 - c. `@Query` solo admite SQL nativo; no permite escribir consultas en lenguaje orientado a objetos.
 - d. `JpaRepository` extiende a `CrudRepository` agregando operaciones adicionales
 - e. Un `Query Method` derivado por nombre puede resolver cualquier consulta. incluyendo agrupaciones y subconsultas, sin necesidad de `@Query`
 - f. En una consulta `@Query` se pueden usar parámetros nombrados (`@Param`) o posicionales (`?1, ?2`)
-

5.

Describir dos situaciones/implementaciones concretas de este modelo donde Spring Data JPA simplifica código en comparación con la implementación manual requerida en Hibernate puro.

6.

Definir la cabecera —interfaz— de un repositorio de una de las tres entidades mapeadas.

```
// Repositorio de una de las entidades mapeadas (Documento, Libro o Tesis)
public interface _____ extends CrudRepository<_____,
_____> {
```

```
// Método/s de consulta (punto siguiente)
```

```
_____
}
```

7.

Sobre ese repositorio, agregar un método que devuelva los documentos escritos por más de n autores. Indicar y justificar la estrategia elegida para escribir la consulta.

MongoDB

8.

Suponiendo dos colecciones: una llamada “**autores**”, donde cada elemento representa a un autor identificado por su `_id`, y otra llamada “**documentos**”, en la que cada elemento referencia a sus autores mediante un arreglo `autorIds`.

Resolver con **Aggregation Framework** por cada autor: calcular la cantidad de documentos en los que participa y el año de publicación promedio de esos documentos; devolver únicamente los autores con más de 10 documentos, ordenados por cantidad de documentos de forma descendente.

9.

Suponiendo que este modelo se mapeara a MongoDB, seleccionar una relación del modelo que convenga modelar como documento embebido y otra que convenga modelar como referencia. Justificar cada una de las elecciones.

Redis

10.

¿Cuáles son los principales casos de uso de Redis en aplicaciones reales? ¿En qué casos conviene usar Redis en lugar de —o junto con— una base de datos relacional y en cuáles no? Proponga un caso de ejemplo.